Atelier DevOps

CI/CD en pratique : construire sa première pipeline

Comprendre l'intégration et le déploiement continus en mettant en place une pipeline automatique sur un petit projet Python, avec GitHub Actions (ou GitLab CI).

Durée	1 séance d'environ 2 h 30 (travail guidé, pas à pas)
Niveau	Débutant : il suffit de savoir utiliser un terminal et les commandes Git de base
Projet	Une mini-calculatrice en Python avec des tests automatiques
Outils	Git, Python 3, un éditeur de code (VS Code), un compte GitHub **ou** GitLab (gratuit)
Résultat	Une pipeline qui s'exécute toute seule à chaque `git push` et que vous pouvez observer sous forme de graphique dans votre navigateur

Objectifs de la séance

- Expliquer avec vos mots ce que sont **CI** (intégration continue) et **CD** (livraison / déploiement continus).
- Écrire un fichier de pipeline et comprendre chacune de ses lignes.
- Lire une pipeline dans l'interface graphique de GitHub ou GitLab (jobs, étapes, logs, artefacts).
- Constater qu'un code cassé est **bloqué automatiquement** avant d'arriver en production, puis le réparer.
- Ajouter une étape de déploiement qui ne s'exécute que sur la branche principale.

Déroulement conseillé

Partie	Contenu	Durée
0	Préparer son poste de travail (Git, Python, compte)	10 min
1	Comprendre le CI/CD (théorie courte, vocabulaire)	15 min
2	Le projet : code, tests et exécution en local	20 min
3	Créer le dépôt distant et envoyer le code	15 min
4	Écrire et déclencher la première pipeline	30 min
5	Observer la pipeline en graphique	15 min
6	Casser le code puis le réparer	15 min
7	Le vrai workflow : branche, Pull Request, fusion et déploiement	20 min
8	Variante GitLab CI et bilan	10 min

Comment utiliser ce document pendant le TP
Chaque partie commence par un encadré orange **À FAIRE** : ce sont les étapes à suivre, dans l'ordre, avec les commandes exactes. Il se termine par un **point de contrôle** (cochez la case quand c'est bon) et une ligne **Si ça bloque**. Le reste de la partie donne les explications et le code à copier.

Partie 0 — Préparer son poste de travail

Avant de commencer, chaque étudiant vérifie que son environnement est prêt. Cette vérification de 10 minutes évite beaucoup de blocages plus tard.

À FAIRE — Partie 0 — étapes à suivre	
1	Ouvrez un **terminal** : PowerShell ou Git Bash sous Windows, Terminal sous macOS et Linux.
2	Vérifiez Git : tapez `git --version`. Une version doit s'afficher (par exemple *git version 2.43.0*).
3	Vérifiez Python : tapez `python --version` (ou `python3 --version` sous macOS et Linux). Il faut Python 3.9 ou plus.
4	Indiquez votre identité à Git (une seule fois par poste) : `git config --global user.name "Prénom Nom"` puis `git config --global user.email "vous@exemple.com"`.
5	Connectez-vous à votre compte **GitHub** (ou **GitLab**) dans le navigateur. Créez-en un gratuitement si besoin.
6	Ouvrez **VS Code** (ou votre éditeur habituel). Rappel : sous macOS et Linux, remplacez `python` par `python3` dans toutes les commandes du TP.
[ ]	**Point de contrôle :** les commandes Git et Python affichent une version, et vous êtes connecté à GitHub ou GitLab.
?	**Si ça bloque :** Git introuvable : installez-le depuis git-scm.com puis rouvrez le terminal. Python introuvable sous Windows : réinstallez-le en cochant « Add Python to PATH ».

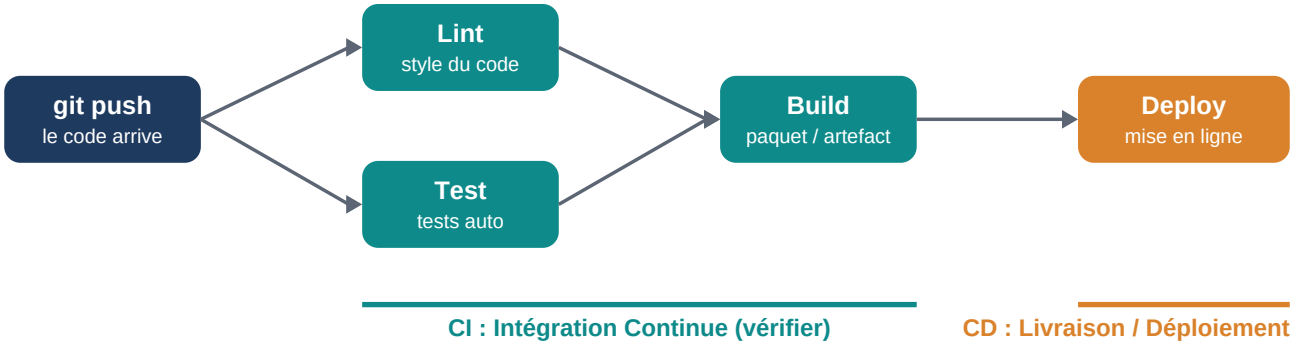
Partie 1 — Comprendre le CI/CD

Le problème que l'on cherche à résoudre

Imaginez une équipe de cinq développeurs qui modifient le même programme. Sans automatisation, chacun teste (ou oublie de tester) à la main, les erreurs se découvrent tard, et mettre le programme en ligne est une opération stressante. Le CI/CD fonctionne comme un **contrôle qualité automatique à la sortie d'une usine** : chaque nouvelle pièce (chaque modification du code) passe par la même chaîne de vérifications, sans exception.

Sigle	Signification	En une phrase
CI	Continuous Integration Intégration continue	À chaque modification, le code est récupéré, vérifié (style, tests) et construit automatiquement.
CD (Delivery)	Continuous Delivery Livraison continue	Le code validé est toujours prêt à être mis en ligne ; l'envoi final se déclenche par un clic humain.
CD (Deployment)	Continuous Deployment Déploiement continu	Si tout est vert, la mise en ligne se fait **toute seule**, sans intervention.

La pipeline que nous allons construire



Chaque case est un **job**. Lint et Test s'exécutent en parallèle ; Build ne démarre que si les deux réussissent ; Deploy ne démarre que si Build réussit. Si un job échoue, la suite est automatiquement stoppée.

Vocabulaire à connaître

Terme	Définition simple
Pipeline	L'enchaînement complet des étapes automatiques déclenché par un événement.
Job	Un bloc de travail (ex. : « lancer les tests ») exécuté sur une machine propre.
Step / script	Une commande ou action à l'intérieur d'un job.
Trigger (déclencheur)	L'événement qui lance la pipeline : un push, une Pull Request, un horaire…
Runner	La machine (virtuelle) fournie par GitHub/GitLab qui exécute vos jobs.
Artefact	Un fichier produit par un job (ex. : le paquet construit) et transmis au job suivant.

À FAIRE — Partie 1 — étapes à suivre	
1	Lisez le schéma de la pipeline ci-dessus en suivant les flèches avec le doigt.
2	Avec votre voisin, expliquez en une phrase chacune de ces notions : **CI**, **livraison continue**, **déploiement continu**.
3	Repérez sur le schéma quels jobs s'exécutent **en même temps** et lequel doit **attendre** les autres.
4	Réfléchissez : « si le job Test échoue, que se passe-t-il pour Build et Deploy ? » Vous vérifierez votre réponse à la Partie 6.
[ ]	**Point de contrôle :** vous savez expliquer pourquoi Deploy est placé en dernier dans la pipeline.
?	**Si ça bloque :** relisez le tableau CI / CD, ou demandez à l'enseignant.

Partie 2 — Le projet : une mini-calculatrice Python

Le projet est volontairement minuscule : l'objectif n'est pas de programmer, mais de comprendre la pipeline. Créez un dossier `calculatrice-cicd` contenant les fichiers ci-dessous.

À FAIRE — Partie 2 — étapes à suivre	
1	Dans le terminal, créez le dossier du projet : `mkdir calculatrice-cicd` puis `cd calculatrice-cicd`.
2	Ouvrez-le dans VS Code : `code .` (ou menu *Fichier* > *Ouvrir le dossier*).
3	Créez les quatre fichiers de l'arborescence ci-dessous : `calculatrice.py`, `test_calculatrice.py`, `requirements.txt` et `.gitignore`.
4	Copiez le contenu de chaque fichier depuis les encadrés **1** à **3** de cette partie, puis sauvegardez (Ctrl+S).
5	Installez la dépendance : `python -m pip install -r requirements.txt`.
6	Lancez les tests : `python -m pytest -v`.
7	*Recommandé :* vérifiez le style comme le fera la pipeline : `python -m pip install flake8` puis `python -m flake8 . --max-line-length=100`. Aucun message ne doit s'afficher.
[ ]	**Point de contrôle :** pytest affiche **5 passed** en vert et flake8 n'affiche rien.
?	**Si ça bloque :** *No module named pytest* : refaites l'étape 5. Aucun test trouvé : le nom du fichier doit commencer par `test_`. Si vous avez créé un environnement virtuel, ajoutez `--exclude=venv` à la commande flake8.

Arborescence du projet

```
calculatrice-cicd/
|-- calculatrice.py          <- le code de l'application
|-- test_calculatrice.py     <- les tests automatiques
|-- requirements.txt         <- les dépendances (pytest)
|-- .gitignore               <- fichiers à ne pas envoyer sur Git
`-- .github/workflows/
    `-- pipeline.yml         <- la pipeline (créée à la partie 4)
```

1 Créer le code de l'application

calculatrice.py

```
def addition(a, b):
    return a + b


def soustraction(a, b):
    return a - b


def multiplication(a, b):
    return a * b


def division(a, b):
    if b == 0:
        raise ValueError("Division par zéro impossible")
    return a / b
```

2 Écrire les tests automatiques

Un test est un petit programme qui vérifie qu'une fonction donne le résultat attendu. C'est lui qui servira de « juge » dans la pipeline.

test_calculatrice.py

```
import pytest
from calculatrice import addition, soustraction, multiplication, division


def test_addition():
    assert addition(2, 3) == 5


def test_soustraction():
    assert soustraction(10, 4) == 6


def test_multiplication():
    assert multiplication(3, 4) == 12


def test_division():
    assert division(10, 2) == 5


def test_division_par_zero():
    with pytest.raises(ValueError):
        division(5, 0)
```

3 Les deux petits fichiers de configuration

requirements.txt

```
pytest
```

.gitignore

```
venv/
__pycache__/
.pytest_cache/
```

4 Lancer les tests sur votre machine

Terminal

```
python -m pip install -r requirements.txt
python -m pytest -v
```

Vous devez voir **5 passed** en vert. Si c'est le cas, votre projet est prêt. Le but de la pipeline sera de refaire *exactement la même chose*, mais automatiquement, sur une machine neutre, à chaque modification.

Partie 3 — Créer le dépôt et envoyer le code

Un **dépôt distant** est la copie de votre projet hébergée sur GitHub (ou GitLab). C'est lui qui déclenchera la pipeline à chaque envoi de code.

À FAIRE — Partie 3 — étapes à suivre	
1	Sur GitHub, cliquez sur **+ > New repository** (sur GitLab : **New project > Create blank project**).
2	Nom du dépôt : `calculatrice-cicd`. Public ou privé, au choix.
3	**Ne cochez rien** : ni README, ni .gitignore, ni licence. Le dépôt doit rester vide. Cliquez sur **Create repository**.
4	Copiez l'adresse HTTPS du dépôt (elle se termine par `.git`).
5	Dans le terminal, **à la racine du projet**, tapez les six commandes du bloc ci-dessous, l'une après l'autre, en remplaçant l'adresse par la vôtre.
6	Si Git demande un mot de passe, collez votre **jeton d'accès personnel** (voir la remarque plus bas).
7	Rafraîchissez la page du dépôt dans le navigateur.
[ ]	**Point de contrôle :** vous voyez vos 4 fichiers sur GitHub : calculatrice.py, test_calculatrice.py, requirements.txt et .gitignore.
?	**Si ça bloque :** *remote origin already exists* : tapez `git remote remove origin` puis refaites la commande `git remote add`. *src refspec main does not match any* : le commit n'a pas été fait, refaites `git add .` puis `git commit`.

Terminal — commandes de l'étape 5

```
git init
git add .
git commit -m "Premier commit : calculatrice et tests"
git branch -M main
git remote add origin https://github.com/VOTRE-NOM/calculatrice-cicd.git
git push -u origin main
```

> **Remarque sur le jeton d'accès**
> Remplacez `VOTRE-NOM` par votre identifiant. Au lieu du mot de passe, GitHub demande un **jeton** (*Settings* > *Developer settings* > *Personal access tokens*). **Important :** pour pouvoir envoyer un fichier de pipeline (Partie 4), le jeton doit avoir la permission **workflow** (jeton classique) ou **Workflows : lecture et écriture** (jeton fine-grained). Vous pouvez aussi utiliser GitHub Desktop ou VS Code, qui gèrent la connexion à votre place.

Partie 4 — Écrire la première pipeline (GitHub Actions)

Sur GitHub, une pipeline est décrite dans un fichier **YAML** placé obligatoirement dans `.github/workflows/`. Avant de l'écrire, voici les mots-clés que vous allez y rencontrer.

À FAIRE — Partie 4 — étapes à suivre	
1	Créez les dossiers de la pipeline : `mkdir -p .github/workflows` (macOS, Linux) ou `mkdir .github\workflows` (Windows). Vous pouvez aussi utiliser clic droit > *Nouveau dossier* dans VS Code. Attention aux noms : **.github** puis **workflows**.
2	Créez le fichier `.github/workflows/pipeline.yml` à l'intérieur.
3	Copiez-y le contenu de la page suivante. **Respectez l'indentation** : 2 espaces par niveau, jamais de tabulation. Sauvegardez.
4	Relisez : vous devez retrouver 4 jobs, nommés `lint`, `test`, `build` et `deploy`.
5	Envoyez le fichier : `git add .github/workflows/pipeline.yml` puis `git commit -m "Ajout de la pipeline CI/CD"` puis `git push`. Ce *push* déclenche la pipeline : vous n'avez rien d'autre à lancer.
6	Sur GitHub, ouvrez l'onglet **Actions**.
[ ]	**Point de contrôle :** une exécution nommée « Pipeline CI/CD Calculatrice » apparaît dans l'onglet Actions (jaune, puis verte).
?	**Si ça bloque :** Rien dans Actions : le fichier n'est pas dans `.github/workflows/` (vérifiez l'orthographe des dossiers). Erreur « workflow scope » au push : votre jeton n'a pas la permission *workflow* (voir Partie 3). Erreur de syntaxe YAML : vérifiez l'indentation et l'absence de tabulations.

Les mots-clés d'un fichier de pipeline

Mot-clé	Rôle
`name`	Le nom affiché dans l'interface graphique.
`on`	Les déclencheurs : ici un push sur *main*, ou une Pull Request vers *main*.
`jobs`	La liste des jobs (les cases du graphique).
`runs-on`	Le type de machine virtuelle qui exécute le job (ici Ubuntu).
`steps`	Les étapes du job, exécutées dans l'ordre.
`uses`	Réutilise une action toute faite (récupérer le code, installer Python, gérer les artefacts).
`run`	Exécute une commande shell, comme dans votre terminal.
`needs`	Impose l'ordre : ce job attend que les jobs listés aient réussi. C'est ce qui dessine les flèches du graphique.
`if`	Condition d'exécution : ici, déployer uniquement depuis la branche *main*.

À vous de jouer : créez le fichier `.github/workflows/pipeline.yml` avec le contenu donné sur la page suivante. Repérez les quatre jobs : **lint, test, build, deploy**.

.github/workflows/pipeline.yml

```
name: Pipeline CI/CD Calculatrice

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint:
    name: Lint (style du code)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install flake8
      - run: flake8 . --max-line-length=100

  test:
    name: Tests automatiques
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest -v

  build:
    name: Construction du paquet
    runs-on: ubuntu-latest
    needs: [lint, test]
    steps:
      - uses: actions/checkout@v4
      - run: mkdir dist && cp calculatrice.py dist/
      - uses: actions/upload-artifact@v4
        with:
          name: calculatrice-paquet
          path: dist/

  deploy:
    name: Déploiement (simulé)
    runs-on: ubuntu-latest
    needs: build
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: calculatrice-paquet
          path: dist/
      - run: ls -l dist/
      - run: echo "Application déployée avec succès !"
```

Partie 5 — Observer la pipeline en graphique

C'est le moment le plus parlant de l'atelier : vous allez voir votre pipeline s'exécuter en direct.

À FAIRE — Partie 5 — étapes à suivre	
1	Sur GitHub, cliquez sur l'onglet **Actions**.
2	Cliquez sur la **dernière exécution** de la liste (elle porte le message de votre commit).
3	Sur la page **Summary**, repérez le **graphique** : les 4 jobs et les flèches. Lint et Test sont-ils bien côte à côte ?
4	Attendez que les 4 jobs soient verts (de quelques secondes à 2 minutes). Notez la **durée totale** affichée.
5	Cliquez sur le job **Tests automatiques**, dépliez l'étape `Run pytest -v` et retrouvez vos 5 tests.
6	Revenez à la page Summary, descendez jusqu'à **Artifacts**, téléchargez `calculatrice-paquet` et ouvrez le fichier .zip : il contient `calculatrice.py`.
7	Cliquez sur le job **Déploiement (simulé)** et lisez le message affiché dans son log.
[ ]	**Point de contrôle :** les 4 jobs sont verts, vous avez lu le log des tests et téléchargé l'artefact.
?	**Si ça bloque :** Job longtemps en attente (*Queued*) : patientez un instant. Pas de graphique : cliquez sur l'exécution elle-même, pas sur le nom du workflow dans la colonne de gauche. Liste vide : reprenez la Partie 4.

1 Ouvrir l'onglet Actions

- Dans votre dépôt GitHub, cliquez sur l'onglet **Actions**.
- Vous voyez la liste des exécutions (*workflow runs*) ; la plus récente porte le message de votre commit.

2 Ouvrir une exécution : le graphique des jobs

- Cliquez sur l'exécution : la page **Summary** affiche un **graphique** avec vos quatre jobs reliés par des flèches, exactement comme le schéma de la partie 1.
- Lint et Test sont côte à côte (parallèles), puis Build, puis Deploy.

3 Lire l'état de chaque job

🟢 Réussi (vert) 🔴 Échoué (rouge) 🟡 En cours (jaune) ⚪ Ignoré / en attente (gris)

4 Consulter les logs et l'artefact

- Cliquez sur le job **Tests automatiques** : vous retrouvez la sortie de `pytest -v` (les 5 tests) comme dans votre terminal.
- Cliquez sur chaque étape pour la déplier et lire le détail.
- En bas de la page Summary, la section **Artifacts** contient *calculatrice-paquet* : le fichier produit par le job Build, téléchargeable.

> **À retenir**
> La pipeline n'est plus une « boîte noire » : chaque étape est visible, chaque commande est tracée, et toute l'équipe voit la même chose.

Partie 6 — Casser le code, puis le réparer

Une pipeline sert surtout quand quelque chose ne va pas. Provoquons une erreur volontaire.

À FAIRE — Partie 6 — étapes à suivre	
1	Ouvrez `calculatrice.py` et, dans la fonction `addition`, remplacez `a + b` par `a - b`. Sauvegardez.
2	Testez d'abord en local : `python -m pytest -v`. Un test doit échouer (rouge) : c'est normal.
3	Envoyez quand même le bug : `git add calculatrice.py`, `git commit -m "Bug volontaire dans l'addition"`, `git push`.
4	Onglet **Actions** : ouvrez la nouvelle exécution. Le job **Tests automatiques** est **rouge**.
5	Cliquez sur ce job, dépliez l'étape en rouge et trouvez le message d'erreur : nom du test (`test_addition`) et valeurs comparées.
6	Revenez au graphique : **notez l'état de Build et de Deploy** (ils ne se sont pas exécutés). Comparez avec votre réponse de la Partie 1.
7	Réparez : remettez `a + b`, vérifiez en local (**5 passed**), puis `git add calculatrice.py`, `git commit -m "Correction de l'addition"`, `git push`.
8	Vérifiez que la nouvelle exécution est entièrement **verte**.
[ ]	**Point de contrôle :** dans la liste des exécutions, vous voyez une exécution rouge suivie d'une exécution verte.
?	**Si ça bloque :** Le job Lint est rouge aussi : vérifiez qu'il y a 2 lignes vides entre les fonctions. Pas de nouvelle exécution : vérifiez que le push a abouti avec `git status`.

1 Introduire un bug

Dans `calculatrice.py`, modifiez la fonction d'addition (un + devient un -) :

calculatrice.py

```
def addition(a, b):
    return a - b      # <-- bug volontaire
```

Terminal

```
git add calculatrice.py
git commit -m "Bug volontaire dans l'addition"
git push
```

2 Observer l'échec

- Retournez dans **Actions** : le job **Tests automatiques** est **rouge**.
- Les jobs **Build** et **Deploy** ne se lancent **pas** : ils restent gris. Le code défectueux n'a donc **jamais** été déployé.
- Ouvrez le job rouge et lisez le log : cherchez la ligne `assert 2 - 3 == 5` ou similaire. Le message vous indique le test, la ligne et la valeur obtenue.

3 Réparer

- Remettez `a + b`, puis `git add`, `commit`, `push`.
- Une nouvelle exécution démarre : tout repasse au **vert** jusqu'à Deploy.

> **Idée clé**
> La pipeline joue le rôle de **filet de sécurité** : l'erreur est détectée en quelques dizaines de secondes, par une machine, avant qu'un utilisateur ne la rencontre.

Partie 7 — Le vrai workflow : branche et Pull Request

Dans une équipe, on ne pousse pas directement sur *main*. On travaille sur une **branche**, puis on propose la modification via une **Pull Request** (PR, appelée *Merge Request* sur GitLab). La pipeline vérifie la PR avant la fusion.

À FAIRE — Partie 7 — étapes à suivre	
1	Créez une branche : `git switch -c ajout-puissance`.
2	À la fin de `calculatrice.py`, ajoutez la fonction `puissance` (2 lignes vides avant).
3	Dans `test_calculatrice.py` : ajoutez `puissance` à la ligne `import`, puis la fonction `test_puissance` (2 lignes vides avant).
4	Lancez `python -m pytest -v` : **6 tests** doivent passer.
5	Envoyez la branche : `git add .`, `git commit -m "Ajout de la fonction puissance"`, `git push -u origin ajout-puissance`.
6	Sur GitHub, cliquez sur **Compare & pull request**, puis sur **Create pull request**.
7	Dans la PR, attendez les **checks** : Lint, Test et Build passent ; **Deploy est ignoré** (*skipped*).
8	Cliquez sur **Merge pull request**, puis **Confirm merge**.
9	Onglet **Actions** : une nouvelle exécution sur *main* démarre ; cette fois **Deploy s'exécute**.
10	Mettez à jour votre copie locale : `git switch main` puis `git pull`.
[ ]	**Point de contrôle :** Deploy est vert après la fusion, et `calculatrice.py` contient la fonction `puissance`.
?	**Si ça bloque :** Pas de bouton *Compare & pull request* : onglet *Pull requests* > *New pull request*, base *main*, compare *ajout-puissance*. Aucun check dans la PR : vérifiez que le YAML contient bien `pull_request`.

1 Créer une branche et ajouter une fonctionnalité

Terminal

```
git switch -c ajout-puissance
```

Ajoutez à la fin de `calculatrice.py` (gardez deux lignes vides avant la fonction) :

calculatrice.py

```
def puissance(a, b):
    return a ** b
```

Ajoutez le test correspondant dans `test_calculatrice.py`, avec les mêmes règles d'espacement. Pensez aussi à compléter la ligne `import` avec `puissance` :

test_calculatrice.py

```
def test_puissance():
    assert puissance(2, 3) == 8
```

Terminal

```
git add .
git commit -m "Ajout de la fonction puissance"
git push -u origin ajout-puissance
```

2 Ouvrir la Pull Request

- GitHub propose un bouton **Compare & pull request** : cliquez dessus puis **Create pull request**.
- En bas de la PR, la section **Checks** montre la pipeline qui s'exécute.
- Observez : Lint, Test et Build s'exécutent, mais **Deploy est ignoré** (gris, « skipped ») car la condition `if` impose la branche *main*.

3 Fusionner et déployer

- Quand les checks sont verts, cliquez sur **Merge pull request**.
- Retournez dans **Actions** : une nouvelle pipeline démarre sur *main*, et cette fois **Deploy s'exécute**.

Le cycle complet du CI/CD
On modifie le code, on le pousse sur une branche, la CI le vérifie, l'équipe le relit et le fusionne, puis la CD le déploie. Tout est automatique, tracé et reproductible.

Partie 8 — Variante GitLab CI

Si vous utilisez GitLab, le principe est identique. Le fichier s'appelle `.gitlab-ci.yml` et se place à la **racine** du projet. Ici, les jobs sont regroupés en **stages** (colonnes du graphique) : les jobs d'un même stage tournent en parallèle, et les stages s'enchaînent dans l'ordre.

À FAIRE — Partie 8 — étapes à suivre

1	Sur GitLab : **New project > Create blank project**, nom `calculatrice-cicd`. Décochez *Initialize repository with a README*.
2	Ajoutez GitLab comme destination : `git remote add gitlab https://gitlab.com/VOTRE-NOM/calculatrice-cicd.git` (adresse copiée depuis *Code* > *Clone with HTTPS*). Si vous travaillez **uniquement** sur GitLab, utilisez directement cette adresse comme *origin* à la Partie 3.
3	Créez le fichier `.gitlab-ci.yml` **à la racine** du projet (pas dans un sous-dossier) avec le contenu ci-dessous.
4	Envoyez : `git add .gitlab-ci.yml`, `git commit -m "Ajout de la pipeline GitLab"`, `git push -u gitlab main`.
5	Menu de gauche : **Build > Pipelines** (**CI/CD > Pipelines** sur les anciennes versions). Cliquez sur la pipeline.
6	Observez les **3 colonnes** (stages) : *verification* (lint + test), *build*, *deploy*. Cliquez sur un job pour lire son log.
7	Comparez avec GitHub grâce au tableau de fin de partie, puis passez au bilan.
[ ]	**Point de contrôle :** la pipeline GitLab est verte avec ses 3 stages visibles dans le graphique.
?	**Si ça bloque :** Pipeline bloquée (*stuck* ou *pending*) : aucun runner n'est disponible. Si GitLab demande de valider votre compte pour utiliser les runners partagés, suivez la procédure proposée, ou restez sur GitHub.

.gitlab-ci.yml

```
image: python:3.12

stages:
  - verification
  - build
  - deploy

lint:
  stage: verification
  script:
    - pip install flake8
    - flake8 . --max-line-length=100

test:
  stage: verification
  script:
    - pip install -r requirements.txt
    - pytest -v

build:
  stage: build
  script:
    - mkdir dist
    - cp calculatrice.py dist/
  artifacts:
    paths:
      - dist/

deploy:
  stage: deploy
  script:
    - ls -l dist/
    - echo "Application déployée avec succès !"
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Voir le graphique sur GitLab

- Dans le menu de gauche, ouvrez **Build > Pipelines** (ou **CI/CD > Pipelines** selon la version de GitLab).
- Cliquez sur le numéro de la pipeline : vous voyez les **stages en colonnes** (verification, build, deploy) et les jobs dans chaque colonne.
- Cliquez sur un job pour lire ses logs ; les artefacts se téléchargent depuis la page du job.

Notion	GitHub Actions	GitLab CI
Fichier de pipeline	.github/workflows/*.yml	.gitlab-ci.yml (à la racine)
Ordre des jobs	needs	stages
Condition de déploiement	if: github.ref == ...	rules: - if: ...
Proposition de modification	Pull Request	Merge Request

Bilan de la séance

Ce que vous avez mis en place

- Un projet Python avec des **tests automatiques**.
- Une pipeline en 4 jobs : **lint, test, build, deploy**.
- Un déclenchement **automatique** à chaque push et à chaque Pull Request.
- Un blocage automatique du déploiement quand un test échoue.
- Une visualisation graphique de la pipeline et de ses logs.

Questions de réflexion (à discuter en groupe)

1. Quelle est la différence entre l'intégration continue et le déploiement continu ?
2. Pourquoi le job Deploy est-il resté gris quand le test était rouge ?
3. Pourquoi lancer le lint et les tests **en parallèle** plutôt que l'un après l'autre ?
4. Que se passerait-il si on retirait la ligne `if: github.ref == 'refs/heads/main'` ?
5. Quels avantages concrets pour une équipe de 10 personnes, comparé à des tests faits « à la main » ?

Pour aller plus loin (facultatif)

- Ajouter un **badge de statut** (vert/rouge) dans le fichier README du dépôt.
- Mesurer la **couverture de tests** avec `pytest-cov`.
- Remplacer le déploiement simulé par un vrai déploiement (GitHub Pages, Render, un serveur…).
- Construire une **image Docker** dans le job Build.
- Recevoir une **notification** (e-mail, Discord, Slack) quand la pipeline échoue.

Aide-mémoire : problèmes fréquents

Symptôme	Cause probable et solution
Rien ne se passe après le push	Le fichier n'est pas dans `.github/workflows/` (ou n'a pas l'extension *.yml*) ; vérifier l'orthographe du dossier.
Erreur de syntaxe YAML	Indentation incorrecte ou tabulation. Utiliser des espaces, 2 par niveau.
Le job Lint est rouge (E302, E305, W292…)	Il manque des lignes vides entre fonctions (2 requises) ou un retour à la ligne en fin de fichier.
`ModuleNotFoundError` dans le job Test	Le fichier *requirements.txt* est absent ou n'a pas été envoyé avec `git add`.
Erreur « refusing to allow a Personal Access Token to create or update workflow »	Le jeton n'a pas la permission *workflow* : en créer un nouveau avec cette permission (voir Partie 3).
`remote origin already exists`	Un dépôt distant est déjà configuré : `git remote remove origin`, puis refaire le `git remote add`.
Onglet Actions vide ou désactivé	Cliquer sur le bouton pour activer les workflows (surtout sur un dépôt forké).
Pipeline GitLab « bloquée » (stuck)	Aucun runner disponible : utiliser les runners partagés de GitLab.com ou en demander l'activation.